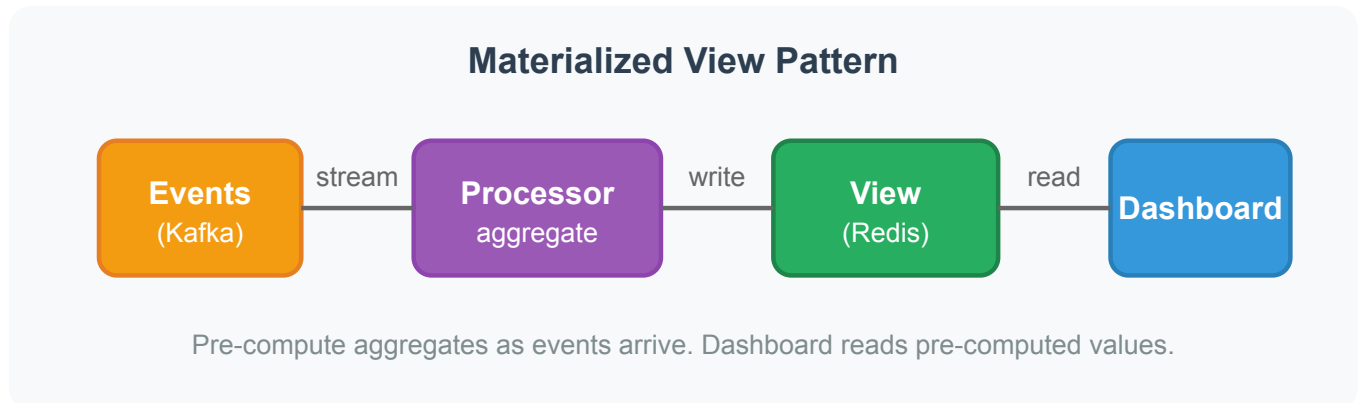


Materialized Views

 systemdesignschool.io/fundamentals/materialized-views-streaming



A dashboard shows page views per minute. The user clicks refresh. The backend runs a query that scans millions of raw events, groups by minute, and returns counts. At low volume, this works. But a busy site generates millions of events daily. Every dashboard refresh scans all that data—and there might be hundreds of concurrent users.

The solution: compute aggregates as events arrive, not when users query. Dashboard reads become key lookups instead of full table scans.

The Problem: Queries That Don't Scale

Consider the straightforward query behind a "views per minute" chart:

```
SELECT minute, COUNT(*) FROM page_events WHERE timestamp > NOW() - INTERVAL '1 hour' GROUP BY minute;
```

For a small site, this completes in milliseconds. But scale changes everything. At 10 million events per day, this query scans roughly 400,000 rows per hour. With 50 concurrent dashboard users refreshing every 10 seconds, the database handles 5 queries per second—each scanning 400,000 rows. That's 2 million row scans per second, just for one chart.

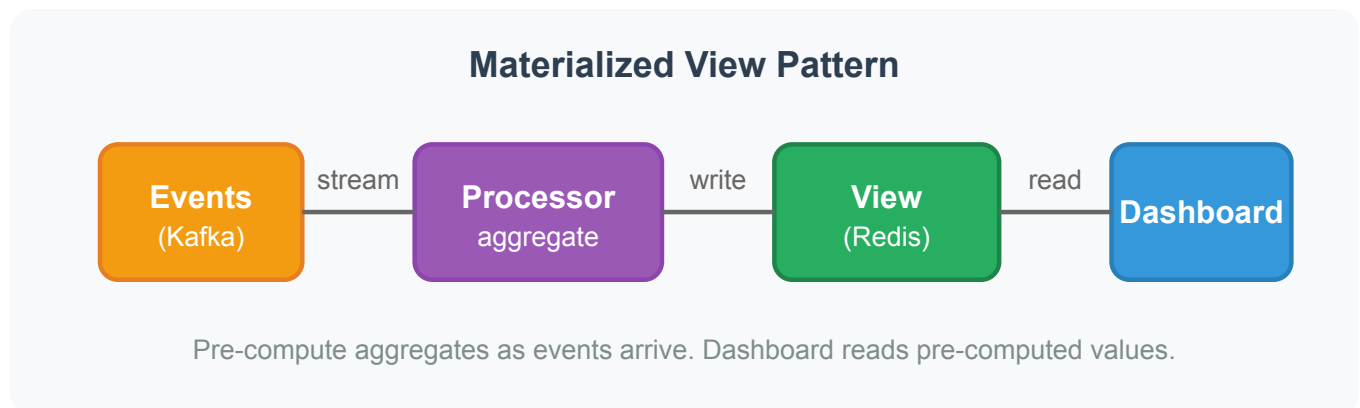
You might think: "Add an index on timestamp." That helps filter to the right hour. But GROUP BY still requires scanning every matching row. The fundamental problem remains: query cost grows with data volume.

Materialized Views: Compute on Write, Not Read

A **materialized view** inverts the work. Instead of computing aggregates when users query, compute them as data arrives. Store the results. Queries become lookups.

A regular database view is just a saved query—it computes on every read. A materialized view stores the computed result. The difference is when work happens: read time vs write time.

For streaming systems, this means the stream processor maintains running aggregates as events flow through. Each event updates the relevant counters. When a user queries, the answer is already waiting.



How It Works: Page Views Example

Let's walk through a concrete implementation. We want instant answers to "page views per minute for the last hour."

Step 1: Events flow from the source. Page view events arrive in Kafka. Each event contains a page ID, user ID, timestamp, and metadata.

Step 2: The stream processor windows events. Using tumbling windows (covered in earlier articles), the processor groups events by (page_id, minute). Each minute boundary triggers output.

Step 3: Window results write to a fast store. When a window closes, the processor writes to Redis:

- Key: `page:{id}:minute:{timestamp}`
- Value: count

Step 4: Dashboard queries the store. To show the last hour, the dashboard fetches 60 keys—one per minute. Each Redis lookup returns in microseconds. Response time is independent of how many events occurred.

The dashboard query transforms from "scan 400,000 rows" to "fetch 60 keys." That's the power of pre-aggregation.

Checkpoint: So far we've moved computation from query time to ingest time. But this creates new challenges. What happens when we need different slices of data? What about late-arriving events?

The Trade-offs

Pre-aggregation isn't free. Understanding the costs helps you decide what to pre-compute.

Storage grows with dimension combinations. Aggregate by page and minute: manageable. Now add region. You need a key for every (page, region, minute) combination. Add device type, browser, experiment variant—keys multiply exponentially.

Consider a site with 10,000 pages, 50 regions, 5 device types, 10 experiment variants. Minute-level aggregates for all combinations would need 25 million keys per minute. That's 1.5 billion keys per hour just for retention. Pre-aggregating every possible slice is impractical.

Late events complicate updates. An event's timestamp says 10:05, but it arrives at 10:08. The 10:05 window closed three minutes ago. What do you do?

Three options:

1. **Ignore late events.** Simple implementation, but counts will be systematically lower than reality. Works if you can tolerate slight inaccuracy.
2. **Update the materialized view.** Correct, but downstream consumers must handle changing values. The "10:05 count" they cached might be stale.
3. **Use wider watermarks.** Wait longer before closing windows. More accurate, but results appear later.

There's no universally right answer—it depends on whether accuracy or timeliness matters more for your use case.

Pre-aggregation locks in query patterns. You decided to aggregate by page and minute. A product manager asks for hourly views by device type. That requires a different aggregation, code changes, and waiting for data to accumulate. Pre-aggregation trades query flexibility for query speed.

Practical Approach: The 80/20 Rule

Pre-aggregate the queries that matter most. Typically, a small number of dashboards account for most of the query load—the operational metrics everyone checks constantly.

For those high-traffic queries, materialized views are worth the complexity. Real-time dashboards showing request rates, error counts, and active users benefit most.

For ad-hoc exploration—"What was the click rate for users in Germany on mobile last Tuesday?"—keep raw events accessible in a separate system. A data warehouse handles these queries with acceptable latency (seconds, not milliseconds). Users running ad-hoc analyses can tolerate that delay.

This two-tier approach gives you the best of both worlds: instant results for the metrics everyone watches, and flexibility for the questions you haven't thought of yet.

What's Next

Materialized views solve the query-time problem by moving computation earlier. From here, **time-series patterns** cover bucketing, rollups, and retention policies for metrics data.